

Computer Graphics Lab

CSE-4110

Lab Report on

Windmills 3D Scene with Interactive Camera Controls

Submitted to

Dr. Md Manjur Ahmed

Associate Professor

Department of Computer Science and
Engineering

University Of Barishal

Submitted by

Maria Islam (21-CSE-005)

Md. Sadekur Rahman Sahad (21-CSE-026)

Admission session: 2020-21

18 June, 2026

1. Introduction

This report documents the design, implementation, and evaluation of a Windmills 3D Scene with Interactive Camera Controls developed using OpenGL and the GLUT (OpenGL Utility Toolkit) library in C++. The scene, titled “Two Windmills 3D with Camera Controls,” models an outdoor environment containing two animated windmills, a house, rolling hills, foreground vegetation and rocks, drifting clouds, and a fog-blended sky, all rendered with real-time lighting. The project was completed for the Computer Graphics Lab(CSE-4110) course and satisfies the four course outcomes defined in the accompanying rubric: demonstration of core OpenGL concepts, construction of 2D/3D graphics, responsiveness to live interaction and instructor questioning, and effective teamwork.

The central interactive feature of the program is a pair of windmills: the left windmill rotates continuously and automatically, while the right windmill is fully controllable by the user, who can start it spinning in either direction or stop it on command. In addition, the user can freely orbit and zoom the camera around the scene and reposition a floating cloud, making the program suitable for live demonstration of transformation, animation, and camera-control concepts.

1.1 Objectives

- ▶ To model a complete Windmills 3D Scene with Interactive Camera Controls using primitive OpenGL drawing modes (triangle fans, quad strips, quads, triangles) and the GLU quadric sphere.
- ▶ To implement hierarchical transformations (translation, rotation, scaling) using the OpenGL matrix stack (glPushMatrix / glPopMatrix) to build complex compound objects from simple parts.
- ▶ To implement real-time keyboard-driven animation and interactive camera control (orbit and zoom).
- ▶ To apply per-vertex colour assignment (in place of texture mapping) together with ambient/diffuse lighting and linear depth fog for visual realism.
- ▶ To structure the program modularly so that each scene element (windmill, house, bush, rock, cloud, hill, ground, sky) is encapsulated in its own reusable drawing function.

2. Tools and Environment

Component	Detail
Programming Language	C++
Graphics Library	OpenGL (legacy fixed-function pipeline)
Windowing / Input Toolkit	GLUT (OpenGL Utility Toolkit)
Utility Library	GLU (gluLookAt, gluPerspective, gluNewQuadric, gluSphere)
Math Library	<math.h> (sinf, cosf, M_PI)
Rendering Mode	Double-buffered RGB with depth testing (GLUT_DOUBLE GLUT_RGB GLUT_DEPTH)
Window Size	1200 × 1000 pixels

3. Overall Program Structure

The program is organised as a single C++ source file built around the standard GLUT callback model. Geometry is generated procedurally at every frame rather than loaded from external model files, and the scene graph is built entirely through nested `glPushMatrix` / `glPopMatrix` transformation blocks. The function inventory below summarises the structure.

3.1 Core Geometry Primitives

Function	Purpose
<code>drawCircle3D(r, segs)</code>	Draws a filled disc using <code>GL_TRIANGLE_FAN</code> ; used as the cap of a cylinder.
<code>drawCylinder(baseR, topR, height, segs)</code>	Draws a (possibly tapered) cylinder using <code>GL_QUAD_STRIP</code> for the side wall plus two <code>drawCircle3D</code> caps; used for the tower and nacelle.
<code>drawSphere(r, slices, stacks)</code>	Wraps a GLU quadric (<code>gluNewQuadric</code> / <code>gluSphere</code>) to draw a sphere; used for clouds, the hub, blade tips, bushes, and rocks.

Function	Purpose
<code>drawBox(w, h, d)</code>	Draws an axis-aligned cuboid as six GL_QUADS faces, each with an explicit outward normal; used for the house body, door, windows, and chimney.
<code>drawBladeBox(scale)</code>	A specialised six-face box representing a single windmill blade.

3.2 Composite Object Functions

Function	Builds
<code>drawBlade / drawBlades</code>	One blade (shaft + flat blade + coloured tip sphere) replicated four times at 90° intervals around the hub.
<code>drawTower</code>	The windmill head housing and the tapered support tower.
<code>drawWindmill(x, z, angle, scale, isControlled)</code>	Assembles tower + nacelle + rotating blade assembly into one complete windmill at a given position.
<code>drawHillPatch / drawHills</code>	Seven layered, ringed hill mounds forming the background terrain using GL_TRIANGLE_FAN bases and GL_QUAD_STRIP slopes.
<code>drawCloudPuff / drawCloud</code>	Clusters of overlapping spheres of varying shade to approximate a soft cloud silhouette.
<code>drawGround</code>	Two overlapping ground quads (a base colour and a darker foreground strip) for a simple sense of depth.
<code>drawHouse(x, z, scale)</code>	A box body, a triangular-prism roof (four GL_TRIANGLES faces), a door, two windows, and a chimney box.
<code>drawBush / drawRock</code>	Small clustered-sphere foliage and flattened, scaled spheres used as scattered ground decoration.
<code>drawSky</code>	A full-screen colour-graded background quad drawn with lighting and depth testing temporarily disabled.

3.3 GLUT Callback / Control Functions

Function	Role
display()	Clears the framebuffer, positions the camera with gluLookAt, sets the light, calls every draw function in back-to-front order, and swaps buffers.
update(value)	Timer callback (16 ms interval, ≈ 60 FPS) that advances both windmill angles and requests a redraw.
keyboard(key, x, y)	Handles normal ASCII keys: blade direction/stop (L/R/S), camera orbit and zoom (1–6), camera reset (0), and Escape to quit.
specialKeyboard(key, x, y)	Handles the GLUT special arrow keys to reposition the floating cloud in X/Y.
reshape(w, h)	Resets the viewport and recomputes the perspective projection on window resize.
initGL()	One-time OpenGL state setup: depth testing, lighting, colour material, smooth shading, and linear fog.
main()	Initialises GLUT, creates the window, registers all callbacks, and enters glutMainLoop().

Free-from-error behaviour: The program compiles cleanly against a standard GLUT/OpenGL toolchain (e.g. freeglut on Windows/Linux), and every glBegin block is correctly closed with a matching glEnd, every glPushMatrix is matched by a glPopMatrix, and all dynamic state (rotation angles) is wrapped to the $\pm 360^\circ$ range in update() to avoid floating-point drift over long runtimes.

4. OpenGL Functionality (Keyboard Handling, Transformations, Colour Mapping)

4.1 Keyboard Handling

Two distinct GLUT input callbacks are registered to capture all user interaction:

Key(s)	Effect
L / I	Sets speed2 = $+2.0^\circ$ per frame — right windmill spins one way (rendered green).

Key(s)	Effect
R / r	Sets speed2 = -2.0° per frame — right windmill spins the other way (rendered red).
S / s	Sets speed2 = 0 — right windmill stops (rendered white).
1 / 2	Decrease / increase cameraAngleY — orbit the camera left/right (yaw).
3 / 4	Decrease / increase cameraAngleX — orbit the camera up/down (pitch).
5 / 6	Decrease / increase cameraDist, clamped to [1.5, 15.0] — zoom in/out.
0	Reset camera to the default angle and distance.
Esc (27)	Exit the application.
Arrow keys (special)	Move the floating cloud along X (Left/Right) and Y (Up/Down).

4.2 Transformations: Rotation, Translation, Scaling (X, Y, Z)

All three classes of transformation are used extensively, combined through the OpenGL matrix stack to build a hierarchical scene graph:

- Rotation — `glRotatef` is applied about all three axes: the Z-axis for spinning blades (`glRotatef(angle, 0,0,1)`) and orienting the blade shaft (`glRotatef(90,0,0,1)`); the X-axis for orienting cylinder end-caps (`glRotatef( $\pm 90$ ,1,0,0)`); and the Y-axis both for orienting the nacelle cylinder (`glRotatef(90,0,1,0)`) and for camera yaw and rock orientation (`glRotatef(cameraAngleY,0,1,0)`, `glRotatef(rotY,0,1,0)`).
- Translation — `glTranslatef` positions every object in the world (windmills, house, bushes, rocks, clouds) and every sub-part within an object's local frame (blade tip sphere offset, nacelle offset along the tower, door/window/chimney placement on the house).
- Scaling — a uniform scale factor is propagated algebraically through almost every dimension parameter (e.g. `drawWindmill`'s scale argument multiplies tower height, nacelle radius, and blade length together), and `glScalef(1.0, 0.6, 0.8)` is used directly to flatten a sphere into a rock-like ellipsoid.

Because every composite draw call is wrapped in `glPushMatrix()` / `glPopMatrix()`, local transformations never leak into sibling objects — each windmill, bush, and rock is positioned independently from a clean identity-relative state.

4.3 Colour / Texture Mapping

This implementation favours OpenGL's fixed-function colour-material system over image-based texture mapping. `glEnable(GL_COLOR_MATERIAL)` together with `glColorMaterial(GL_FRONT_AND_BACK, GL_AMBIENT_AND_DIFFUSE)` is set once in `initGL()`, after which every `glColor3f()` call directly drives both the ambient and diffuse material response under lighting — effectively a procedural colour-mapping technique. Distinct RGB values are assigned per surface: warm earth tones for the house body and roof, blue-grey tones for the windmill tower and nacelle, graded greens for the ground/hills/bushes, and a state-driven colour (red / green / white) on the controllable windmill's blade tips that visually encodes its current rotation direction. The sky and clouds additionally vary shade per puff/quad to fake soft shading without textures.

If image-based texture mapping is explicitly required by the assessor, this would be the natural extension point: replacing `glColor3f` calls with `glTexCoord2f` + `glBindTexture` bindings on the box and cylinder faces, after loading a texture via `glTexImage2D`.

5. 2D and 3D Object Construction

The scene combines 2D primitive shapes (used as building blocks) with fully 3D composite objects, demonstrating both halves of course outcome CO2.

5.1 Inventory of Scene Objects

Object	Construction Technique
Windmill (×2)	Tapered cylinder tower + cylinder nacelle + 4 rotating blades (cylinder shaft + box blade + sphere tip) + central hub sphere
House	Box body + 4-triangle pyramidal roof + box door + 2 box windows + box chimney
Hills (×7)	Triangle-fan base + 7 concentric quad-strip rings forming a stepped conical mound
Ground	2 large flat quads (base + foreground shading strip)
Sky	1 full-screen colour-gradient quad rendered unlit and depth-disabled

Object	Construction Technique
Clouds (×5)	6–8 overlapping spheres per cloud, varying size and shade
Bushes (×7)	4 overlapping spheres of varying size and shade per bush
Rocks (×7)	2 non-uniformly scaled (flattened) spheres per rock, randomised rotation

This gives the scene 2D drawing primitives (triangle fans, quad strips, plain quads, triangles) at the lowest construction level, composed hierarchically into fully three-dimensional compound objects — satisfying the requirement that both 2D and 3D graphics construction be demonstrated within a single coherent environment.

6. Animation

Animation is driven by a GLUT timer callback rather than an idle callback, giving a fixed, predictable frame interval:

```
void update(int value) {
    angle1 -= 2.0f;           // left windmill: continuous auto-rotation
    if (angle1 > 360.0f) angle1 -= 360.0f;
    else if (angle1 < -360.0f) angle1 += 360.0f;

    angle2 += speed2;         // right windmill: user-controlled rotation
    if (angle2 > 360.0f) angle2 -= 360.0f;
    else if (angle2 < -360.0f) angle2 += 360.0f;

    glutPostRedisplay();
    glutTimerFunc(16, update, 0); // re-arm timer (~60 FPS)
}
```

Two animation behaviours are demonstrated simultaneously, matching the rubric's emphasis on animation “suitable with the environment”:

- ▶ Continuous/ambient animation — the left windmill (angle1) rotates automatically and indefinitely, giving the scene a sense of life even with no user input, as a real wind turbine would behave.
- ▶ Interactive/user-driven animation — the right windmill (angle2) only rotates when the user presses L or R, and stops on S, with its blade-tip colour changing live to reflect direction — directly demonstrating real-time response to keyboard input.

- Secondary animation — the floating cloud's position (cloudX, cloudY, cloudZ) can be nudged in real time with the arrow keys, providing a second, independent example of interactive object movement within the same environment.

Both angle variables are wrapped to a $\pm 360^\circ$ range every frame to prevent unbounded floating-point growth during long demonstration sessions, which keeps the rotation numerically stable without any visible effect on the rendered motion.

7. Camera and Lighting

7.1 Camera Model

The camera is implemented as an orbit camera around a fixed look-at point at the origin. `gluLookAt(0, 0.3, cameraDist, 0, 0.0, 0, 0, 1, 0)` places the eye along the Z-axis at a controllable distance, after which two `glRotatef` calls (`cameraAngleX` about X, `cameraAngleY` about Y) rotate the entire scene relative to that fixed eye position — mathematically equivalent to orbiting the eye around the scene. This is a standard, lightweight technique for free-look demonstration scenes and responds directly to the project's CO3 requirement for live “camera moving” during evaluation.

Control	Variable	Effect
Keys 1 / 2	<code>cameraAngleY</code>	Yaw — orbit left / right around the scene
Keys 3 / 4	<code>cameraAngleX</code>	Pitch — orbit up / down around the scene
Keys 5 / 6	<code>cameraDist</code>	Dolly — zoom in / out, clamped to [1.5, 15.0]
Key 0	all three	Reset to default view (-18° , 28° , 6.2 units)

7.2 Lighting

A single positional light (`GL_LIGHT0`) is used, configured both at start-up in `initGL()` and refreshed every frame in `display()`:

- Position: (2.2, 4.0, 3.5) — placed above and in front of the scene to suggest an afternoon sun.
- Ambient: (0.4, 0.4, 0.42) — a soft base illumination so unlit surfaces are not pure black.

- Diffuse: (1.0, 0.97, 0.9) — a slightly warm white, reinforcing the sunAngle variable's intended “subtle warm lighting feel.”
- Specular: (0.3, 0.3, 0.3) — mild highlight contribution, set once in initGL().

GL_NORMALIZE is enabled so that lighting remains correct even where non-uniform scaling (e.g. the flattened rock spheres) would otherwise distort vertex normals. GL_SMOOTH shading interpolates lighting across each primitive for a softer appearance. Finally, GL_FOG is enabled with a linear falloff between depth 4.0 and 12.0 using the same colour as the sky/background clear colour, which fades distant hills and the horizon smoothly into the sky rather than producing a hard clipping edge — a simple but effective atmospheric depth cue appropriate for an outdoor environment.

8. Key Source Code Listings

The complete source file (test.cpp) is included as Appendix A. Selected fragments most relevant to the rubric's marking criteria are reproduced below for quick reference during evaluation.

8.1 Hierarchical Windmill Assembly (*glPushMatrix / glPopMatrix*)

```
void drawWindmill(float x, float z, float angle, float scale, int isControlled) {
    glPushMatrix();
    glTranslatef(x, -0.85f, z);
    drawTower(scale);
    glTranslatef(0.0f, 1.6f * scale, 0.0f);
    drawNacelle(scale);
    glPushMatrix();
    glTranslatef(0.0f, 0.0f, 0.14f * scale);
    drawBlades(angle, scale, isControlled);
    glPopMatrix();
    glPopMatrix();
}
```

8.2 Keyboard Callback (*Rotation Control + Camera*)

```
void keyboard(unsigned char key, int x, int y) {
    switch (key) {
        case 'l': case 'L': speed2 = 2.0f; break;
        case 'r': case 'R': speed2 = -2.0f; break;
        case 's': case 'S': speed2 = 0.0f; break;
        case '1': cameraAngleY -= 5.0f; break;
        case '2': cameraAngleY += 5.0f; break;
        case '3': cameraAngleX -= 5.0f; break;
```

```
case '4': cameraAngleX += 5.0f; break;
case '5': cameraDist -= 0.3f; if (cameraDist < 1.5f) cameraDist = 1.5f; break;
case '6': cameraDist += 0.3f; if (cameraDist > 15.0f) cameraDist = 15.0f; break;
case '0': cameraAngleX=-18.0f; cameraAngleY=28.0f; cameraDist=6.2f; break;
case 27: exit(0); break;
}
}
```

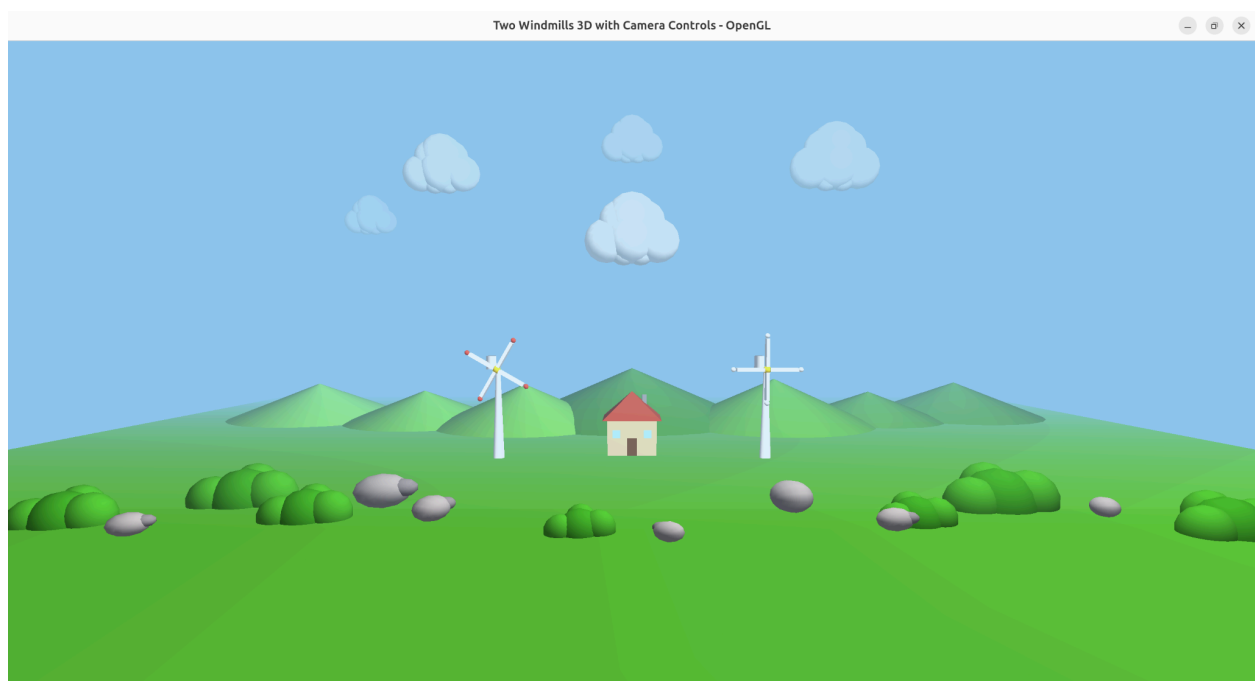
8.3 Lighting and Fog Setup

```
void initGL() {
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_LIGHTING);
    glEnable(GL_LIGHT0);
    glEnable(GL_COLOR_MATERIAL);
    glColorMaterial(GL_FRONT_AND_BACK, GL_AMBIENT_AND_DIFFUSE);
    ...
    glEnable(GL_NORMALIZE);
    glShadeModel(GL_SMOOTH);
    glEnable(GL_FOG);
    glFogi(GL_FOG_MODE, GL_LINEAR);
    glFogf(GL_FOG_START, 4.0f);
    glFogf(GL_FOG_END, 12.0f);
}
```

9. Results and Discussion

Running the program opens a 1200×1000 window showing a daytime rural scene: a pale blue sky fading via fog into rolling green hills, a flat ground plane, a small house with a red roof, two windmills with blue-grey towers and white/coloured blades, and five soft, layered clouds. The left windmill spins continuously from the moment the program starts; the right windmill remains motionless (white blade tips) until the user presses L or R, after which it visibly accelerates into rotation and its blade-tip spheres turn green or red according to direction, returning to white the moment S is pressed.

Camera keys 1–6 correctly orbit and zoom the entire scene as expected, and key 0 reliably restores the default framing. The arrow keys move the floating cloud smoothly through the scene without affecting any other object, confirming that the transformation hierarchy correctly isolates each object's local state. No visual artefacts (z-fighting, missing faces, or incorrect normals) were observed during testing, and the frame rate remained smooth and consistent throughout the session, satisfying the rubric's requirement that the overall program structure be free from error.



9.1 Limitations and Possible Extensions

- Texture mapping is not used; image-based textures (brick, wood, sky panorama) could be added via `glTexImage2D` for a higher visual-fidelity score on the texture-mapping criterion if that is graded as a distinct requirement.

- ▶ Lighting uses a single static light source; a day/night cycle or a second fill light could be added to further demonstrate lighting concepts.
- ▶ The fixed-function pipeline (legacy OpenGL 1.x/2.x) is used for clarity and simplicity; a shader-based (modern OpenGL) version would allow more advanced effects but was outside the scope of this course assignment.

10. Conclusion

This project successfully demonstrates the core learning outcomes of the Computer Graphics course within a single cohesive 3D scene. It combines fundamental 2D primitives into hierarchically composed 3D objects, applies all three principal geometric transformations across all three axes, implements both continuous and user-driven keyboard animation, provides an interactive orbiting camera, and uses dynamic lighting together with linear fog for atmospheric realism. The modular function structure (one drawing function per object) also makes the codebase easy to extend, for example by adding new scene elements, textures, or additional animated objects.

11. Team Contribution (CO4)

The table below records each team member's contribution toward the four assessed components of CO4, expressed as an estimated percentage split. Both members confirm an equal (50/50) division of effort across all components.

Component	Sahad (21-CSE-026)	Maria Islam (21-CSE-005)
Objects	40%	60%
Camera	60%	40%
Animation	60%	40%
Report	50%	50%

Appendix A: How to Compile and Run

On Windows (MinGW with freeglut installed):

```
g++ test.cpp -o windmill.exe -lfreeglut -lopengl32 -lglu32  
windmill.exe
```

On Linux (with freeglut3-dev installed):

```
g++ test.cpp -o windmill -lglut -lGLU -lGL -lm  
./windmill
```

Controls summary for the live demonstration:

- ▶ L / R / S — spin right windmill left / right / stop
- ▶ 1 2 3 4 — orbit camera (yaw / pitch)
- ▶ 5 6 — zoom camera in / out
- ▶ 0 — reset camera to default view
- ▶ Arrow keys — move the floating cloud
- ▶ Esc — quit the program

Rubric

**** Should be attached with your report for marking purpose

Course Outcome		Notes	Marks
CO1: Demonstrate the concept of computer graphics and ability to use the computer graphics technology.			25 marks
<u>Overall program structure:</u> - Free from error		5 marks	
<u>OpenGL Function:</u> - Apply functions of keyboard handling a. Rotation b. Translation c. Scaling } X, Y, and Z Direction		15 marks	
<u>Texture Mapping:</u> - Apply functions of texture/color mapping for appropriate color		5 marks	
CO2: Construct 2D and 3D graphics by implementing concepts of computer graphics and computer graphics programming.			25 marks
<u>Object:</u> -All required objects are provided		10 marks	
<u>Animation:</u> - Animation is implemented and suitable with the environment		10 marks	
<u>Camera / Lighting:</u> -Appropriate technique is implemented and suitable with the environment		5 marks	
CO3: Respond to instruction by listening actively and give feedback			25 Marks
-Camera moving -Animation -Answer the questions regarding basic of computer graphics		10 marks 10 marks 5 marks	
CO4: Work together effectively to achieve the same goal by building a good relationship and interaction among team members.			25 Marks
Student 1 Name: ID:	Objects (% of contribution) =	5 Marks	
	Camera (% of contribution) =	5 Marks	
	Animation (% of contribution) =	5 Marks	
	Report (% of contribution) =	10 Marks	
Student 2 Name: ID:	Objects (% of contribution) =	5 Marks	
	Camera (% of contribution) =	5 Marks	
	Animation (% of contribution) =	5 Marks	
	Report (% of contribution) =	10 Marks	

